

Практическая работа №3-4

Типы данных, арифметика и корректный расчёт показателей

Введение

Исходный проект намеренно содержит незавершённые места и отдельные спорные решения. Результат оценивается не только по факту запуска программы, но и по способности объяснить алгоритм, класс, метод, проверку входных данных и изменение требования на защите.

Цель работы состоит в закреплении конкретного набора средств Java на прикладной задаче и в формировании привычки связывать программный код с алгоритмической и объектной моделью. Студенту необходимо доработать решение и получить функциональное приложение, и при этом сохранить понятную структуру программы, корректную обработку данных и соответствие между Activity-диаграммой, диаграммой классов и фактической реализацией.

1. Осваиваемые элементы Java, которые необходимо использовать:

- примитивные типы и String
- объявление переменных и констант
- арифметические операции и порядок вычислений
- Scanner и форматированный вывод
- простая проверка диапазонов
- выделение повторяемого расчёта в метод

2. Предметная постановка

Отдел закупок получает предварительные параметры заказа и должен рассчитать базовую стоимость, скидку, НДС и итоговую сумму. На данном этапе программа остаётся небольшой и намеренно не разбивается на доменную модель. Основная задача

состоит в том, чтобы научиться выбирать типы данных, выполнять вычисления без скрытых ошибок округления и проверять входные значения до использования.

3. Обязательные правила, которые нужно учитывать для корректного выполнения задания:

- Количество единиц должно быть больше нуля и не превышать 10 000.
- Цена одной единицы должна быть положительной и не превышать 5 000 000 рублей.
- Скидка задаётся в процентах от 0 до 30.
- НДС вычисляется после применения скидки.
- При ошибке входного значения программа должна вывести понятное сообщение и не выполнять расчёт.

4. Activity-диаграмма

На рисунке 1 дан каркас алгоритма. Красные блоки TODO намеренно оставлены неполными. Напишите конкретные действия, которые вам нужно сделать, чтобы проверить корректность и правильность работы проекта. Формулировка в блоке на диаграмме должна отражать конкретную проверку или преобразование, а не общее выражение вроде «*обработать данные*».

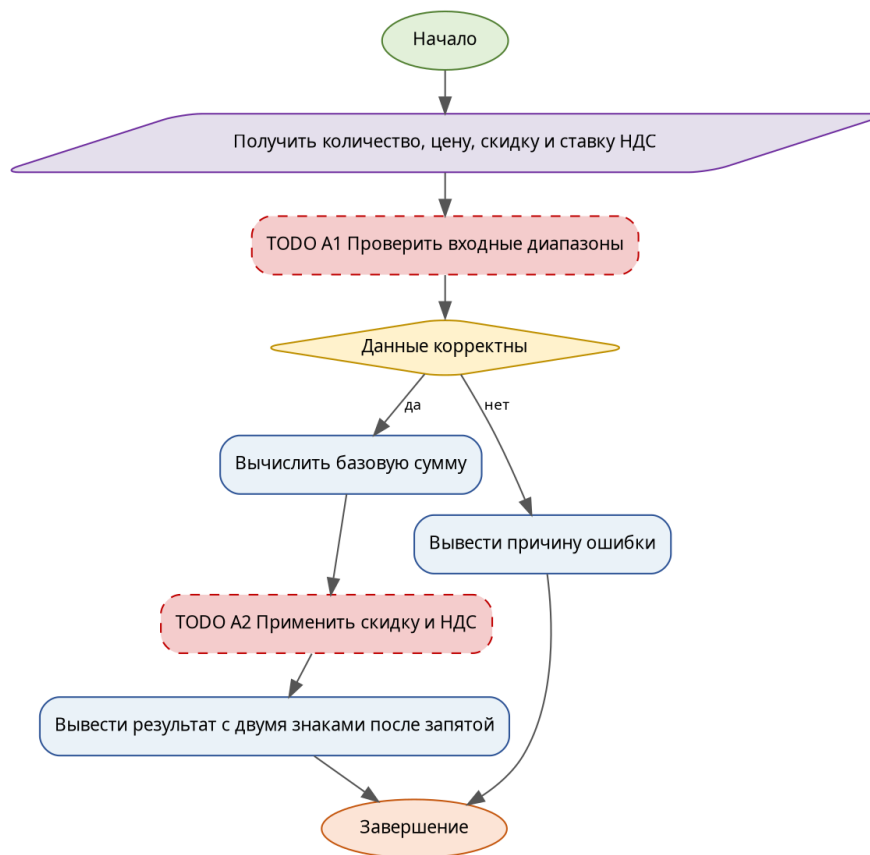


Рисунок 1 — Activity-диаграмма для практической работы 1

5. Диаграмма классов

Диаграмма классов описывает логику и структуру разрабатываемого будущего решения. Все элементы, где имеется пометка TODO необходимо довести до состояния, соответствующего итоговому коду. Если в процессе выполнения вы обоснованно изменяете состав классов или зависимостей, то **не забудьте обновить итоговую диаграмму**. Наличие класса на рисунке 2, которого нет в проекте, и наличие существенного класса в проекте, отсутствующего на диаграмме, **считается несогласованностью**.

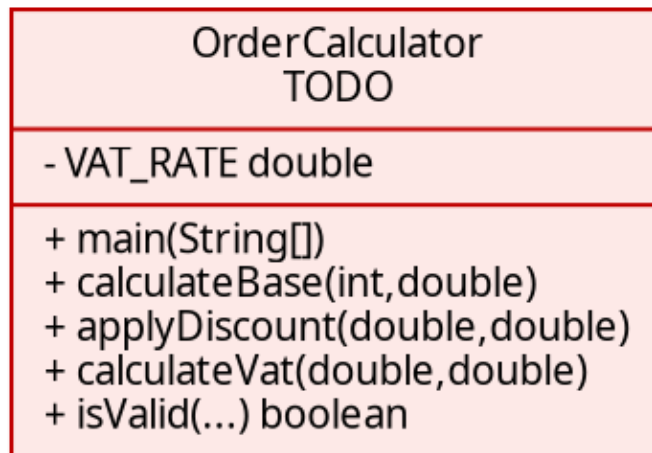


Рисунок 2 — Неполная диаграмма классов для практической работы 1

6. Исходный проект

В каталоге `starter_project` находится компилируемый, но незавершённый Java-проект. **TODO** в исходном коде являются частью задания. Не требуется сохранять **TODO** в итоговой версии. Допускается добавление `private` методов и небольших вспомогательных типов, если это делает решение понятнее. Удалять `SelfCheck` до завершения работы нельзя. Основной акцент вы должны сделать на правильной логике, границах условий и аккуратной декомпозиции методов.

7. Последовательность выполнения, которые необходимо сделать в проекте:

1. Дополнить проверку количества, цены и скидки.
2. Реализовать методы вычисления базовой суммы, скидки и НДС.
3. Исключить повторение одной и той же формулы в `main`.
4. Оформить вывод в рублях с двумя знаками после запятой.
5. Дополнить Activity-диаграмму точными формулировками A1 и A2.
6. Пояснить выбор типов `int` и `double` и указать ограничение `double` для денежных расчётов.

8. Фрагмент кода, который необходимо проанализировать

Ниже приведён короткий фрагмент, который выглядит правдоподобно, но нарушает правило текущей работы. Необходимо объяснить проблему и показать исправленный подход в своём проекте или отчёте.

```
double total = price * quantity - discount; // discount здесь ошибочно  
трактруется как абсолютная сумма
```

Сформулируйте конкретную ошибку этого фрагмента применительно к текущему проекту

9. Входные данные и граничные проверки

В *starter_project/data* находится открытый набор *sample.csv*. Он не является полным тестом. Вам нужно добавить собственные проверки. На защите используется другой набор входных данных, поэтому решение, подогнанное только под открытый файл, не считается завершённым.

Проверка	Что необходимо подтвердить
корректное типовое значение	ожидаемый успешный результат
нижняя граница	проверка точного включения или исключения
верхняя граница	проверка точного включения или исключения
пустое или отсутствующее значение	контролируемая ошибка
значение неверного формата	контролируемая ошибка
повтор или конфликт данных	поведение согласно правилам работы

10. Индивидуальная часть

Номер варианта вычисляется как остаток от деления двух последних цифр номера зачётной книжки на 8 плюс 1. Вариант изменяет одно локальное правило, поэтому студент должен заранее определить место в коде, где параметр или правило должно быть сосредоточено. Размножение одного и того же порога по нескольким классам усложнит защиту.

Вариант	Изменяемый аспект	Параметр
1	ставка НДС	значение или модификатор 10 точное применение определяется текстом текущего правила
2	максимальная скидка	значение или модификатор 15 точное применение определяется текстом текущего правила
3	порог бесплатной обработки	значение или модификатор 20 точное применение определяется текстом текущего правила
4	ставка НДС	значение или модификатор 25 точное применение определяется текстом текущего правила
5	максимальная скидка	значение или модификатор 30 точное применение определяется текстом текущего правила
6	порог бесплатной обработки	значение или модификатор 35 точное применение определяется текстом текущего правила

7	ставка НДС	значение или модификатор 40 точное применение определяется текстом текущего правила
8	максимальная скидка	значение или модификатор 45 точное применение определяется текстом текущего правила

Запишите номер своего варианта и точное правило, которое вы реализовали

11. Собственные формулировки студента

Какая проверка в этой работе защищает программу от наиболее вероятной ошибки входных данных и где она расположена

Какой метод или класс содержит ключевое бизнес-правило и почему вы разместили его именно там

Как Activity-диаграмма отражает один конкретный участок вашего кода

Какое изменение требования в этой работе должно затронуть минимальное количество классов и почему

12. Что необходимо сдать

- исходный проект, который компилируется без удаления файлов
- реализованные TODO и собственные методы, необходимые для решения
- обновлённую Activity-диаграмму
- обновлённую диаграмму классов
- заполненные собственные формулировки из задания
- результаты минимум пяти граничных проверок
- краткий файл DECISIONS.md с двумя решениями, которые студент считает принципиальными
- проект должен быть загружен в любую систему контроля версий и предоставлен в виде ссылки на решения

13. Критерии оценивания

Компонент	Вес	Признак завершённости
Корректность программы	35 %	правила и граничные случаи выполняются
Структура кода	20 %	ответственность не сосредоточена в одном методе
Диаграммы	15 %	совпадают с фактической реализацией

Проверки и устойчивость	15 %	ошибочные данные обрабатываются предсказуемо
Собственные объяснения и защита	15 %	студент объясняет и локально изменяет решение

14. Справочная информация

При выполнении работы рекомендуется сверять синтаксис и поведение библиотек с официальной документацией. Использовать вторичный источник вместо проверки API допустимо только как дополнение.

- Спецификация Java SE 25
- Учебные материалы Java Language Basics
- Классы и объекты Java
- ООП, интерфейсы и наследование